LAPORAN PRAKTIKUM MATA KULIAH

METODE NUMERIK

PRAKTIKUM 07– SPNL



DISUSUN OLEH:

L0325021 – DUTA LATHIIF HAQIM

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA

UNIVERSITAS SEBELAS MARET

2026

BAB I

ANALISIS SOURCE CODE

1.Analisis Source Code “GRAFIK GAMBAR PERSAMAAN 01”

```
f  = @(x) 3*x.^3 - 5*x.^2 + 4*x - 8;
g1 = @(x) 3*x.^3 - 5*x.^2 + 4*x - 7;
g2 = @(x) ones(size(x)) * 1;

% Rentang sumbu x
xmin = -1;
xmax = 3;

% Judul dan keterangan
judul = 'Persamaan 1';
ket1  = '3x^3 - 5x^2 + 4x - 7';
ket2  = 'y = 1';
% ======================================================================

x = linspace(xmin, xmax, 1000);

% ---------------------- GRAFIK TUNGGAL ----------------------
figure(1);
plot(x, f(x), 'b-', 'LineWidth', 2); hold on;
plot(x, zeros(size(x)), 'k--', 'LineWidth', 1);
grid on;
xlabel('x'); ylabel('f(x)');
title([judul ' - Grafik Tunggal']);
legend('f(x)', 'y = 0', 'Location', 'best');

% ---------------------- GRAFIK GANDA ----------------------
figure(2);
plot(x, g1(x), 'b-', 'LineWidth', 2); hold on;
plot(x, g2(x), 'r-', 'LineWidth', 2);
grid on;
xlabel('x'); ylabel('y');
title([judul ' - Grafik Ganda']);
legend(ket1, ket2, 'Location', 'best');
```

i. Analisis source code

Program ini bertujuan menggambar kurva fungsi dengan metode grafik tunggal dan grafik ganda untuk memperkirakan letak akar persamaan nonlinear $3x^3 - 5x^2 + 4x - 7 = 1$.

Rumus yang dihitung:

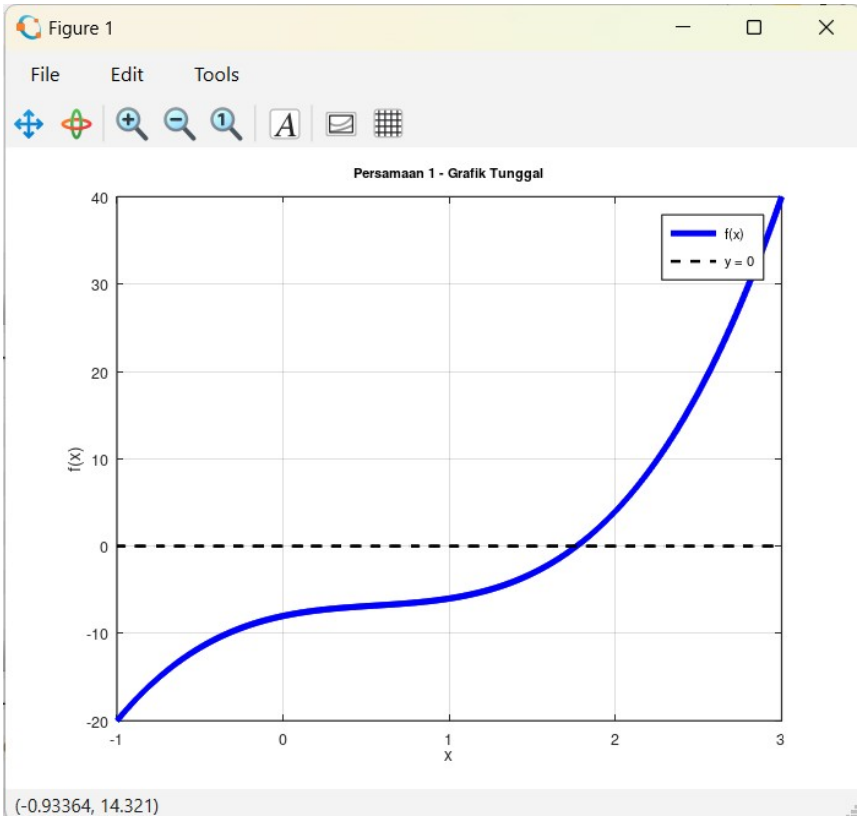
- Grafik tunggal: akar dicari dari f(x) = 0, dengan $f(x) = 3x^3 - 5x^2 + 4x - 8$ (hasil memindahkan konstanta 1 ke ruas kiri)
- Grafik ganda: akar dicari dari $g_1(x) = g_2(x)$, dengan $g_1(x) = 3x^3 - 5x^2 + 4x - 7$ dan $g_2(x) = 1$

Penjelasan garis besar:

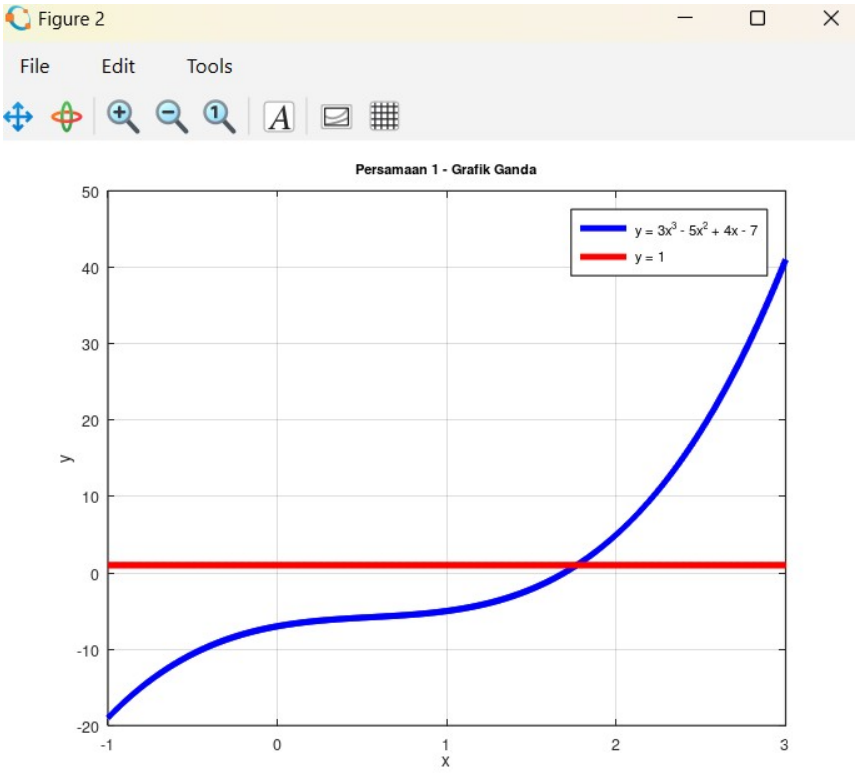
1. Bagian INPUT mendefinisikan fungsi f, g1, dan g2 sebagai anonymous function (@(x)). Operator .^ dipakai agar fungsi dapat menerima vektor x. Fungsi g2 ditulis ones(size(x))*1 agar konstanta y = 1 menghasilkan vektor yang ukurannya sama dengan x.
2. Rentang sumbu x ditentukan oleh xmin = -1 dan xmax = 3. Variabel judul, ket1, dan ket2 mengatur judul dan legend grafik.

3. Perintah x = linspace(xmin, xmax, 1000) membentuk 1000 titik x berjarak sama pada selang [−1, 3].
4. Figure 1 (grafik tunggal) menggambar f(x) sebagai garis biru tebal dan y = 0 sebagai garis putus-putus hitam. Akar adalah titik tempat kurva memotong garis y = 0.
5. Figure 2 (grafik ganda) menggambar $g_1(x)$ sebagai garis biru dan $g_2(x)$ sebagai garis merah. Akar adalah absis titik perpotongan kedua kurva.
6. Kedua grafik diberi grid on, label sumbu, judul, dan legend.

ii. Analisis hasil



Grafik tunggal: kurva $f(x) = 3x^3 - 5x^2 + 4x - 8$ pada selang [−1, 3] naik dari f(−1) = −20 menuju f(3) = 40. Nilai fungsi negatif untuk x kecil (f(0) = −8, f(1) = −6), lalu berubah tanda menjadi positif di antara x = 1 dan x = 2 (f(2) = 4). Kurva memotong sumbu x di sekitar x ≈ 1,77. Perubahan tanda yang lebih rinci mendukung hal ini: f(1,76) ≈ −0,09 dan f(1,77) ≈ +0,05. Hanya ada satu titik potong karena turunannya, $f'(x) = 9x^2 - 10x + 4$, selalu positif (diskriminan 100 − 144 < 0), sehingga fungsi selalu naik.



Grafik ganda: kurva $g_1(x) = 3x^3 - 5x^2 + 4x - 7$ naik dari $g_1(-1) = -19$ sampai $g_1(3) = 41$, dan memotong garis horizontal $y = 1$ di titik yang sama, yaitu $x \approx 1{,}77$. Pada x tersebut $g_1(x) \approx 1$, sehingga $g_1(x) = g_2(x)$ terpenuhi. Hasilnya konsisten dengan grafik tunggal, karena $f(x)$ hanyalah $g_1(x) - g_2(x)$. Bentuk kedua kurva sama persis, dan perbedaannya hanya pada posisi vertikal: grafik tunggal bergeser 1 satuan ke bawah, sehingga akarnya terbaca pada $y = 0$.

Kesimpulan: kedua metode grafik memberikan akar yang sama, yaitu $x \approx 1{,}77$, dan akar ini satu-satunya pada selang $[-1, 3]$. Nilai ini hanya perkiraan kasar yang dapat dipakai sebagai selang awal [1,7; 1,8] atau tebakan awal pada metode numerik seperti bisection dan Newton-Raphson.

2. Analisis Source Code “GRAFIK GAMBAR PERSAMAAN 02”

```
f  = @(x) exp(x) - x.^2;
g1 = @(x) exp(x);
g2 = @(x) x.^2;



% Rentang sumbu x
xmin = -2;
xmax = 2;

% Judul dan keterangan
judul = 'Persamaan 2';
ket1  = 'y = e^x';
ket2  = 'y = x^2';
% ========================================================================

x = linspace(xmin, xmax, 1000);

% ---------------------- GRAFIK TUNGGAL ----------------------
figure(1);
plot(x, f(x), 'b-', 'LineWidth', 2); hold on;
plot(x, zeros(size(x)), 'k--', 'LineWidth', 1);
grid on;
xlabel('x'); ylabel('f(x)');
title([judul ' - Grafik Tunggal']);
legend('f(x)', 'y = 0', 'Location', 'best');

% ---------------------- GRAFIK GANDA ----------------------
figure(2);
plot(x, g1(x), 'b-', 'LineWidth', 2); hold on;
plot(x, g2(x), 'r-', 'LineWidth', 2);
grid on;
xlabel('x'); ylabel('y');
title([judul ' - Grafik Ganda']);
legend(ket1, ket2, 'Location', 'best');
```

i. Analisis source code

Program ini bertujuan menggambar kurva fungsi dengan metode grafik tunggal dan grafik ganda untuk memperkirakan letak akar persamaan nonlinear $e^x - x^2 = 0$.

Rumus yang dihitung:

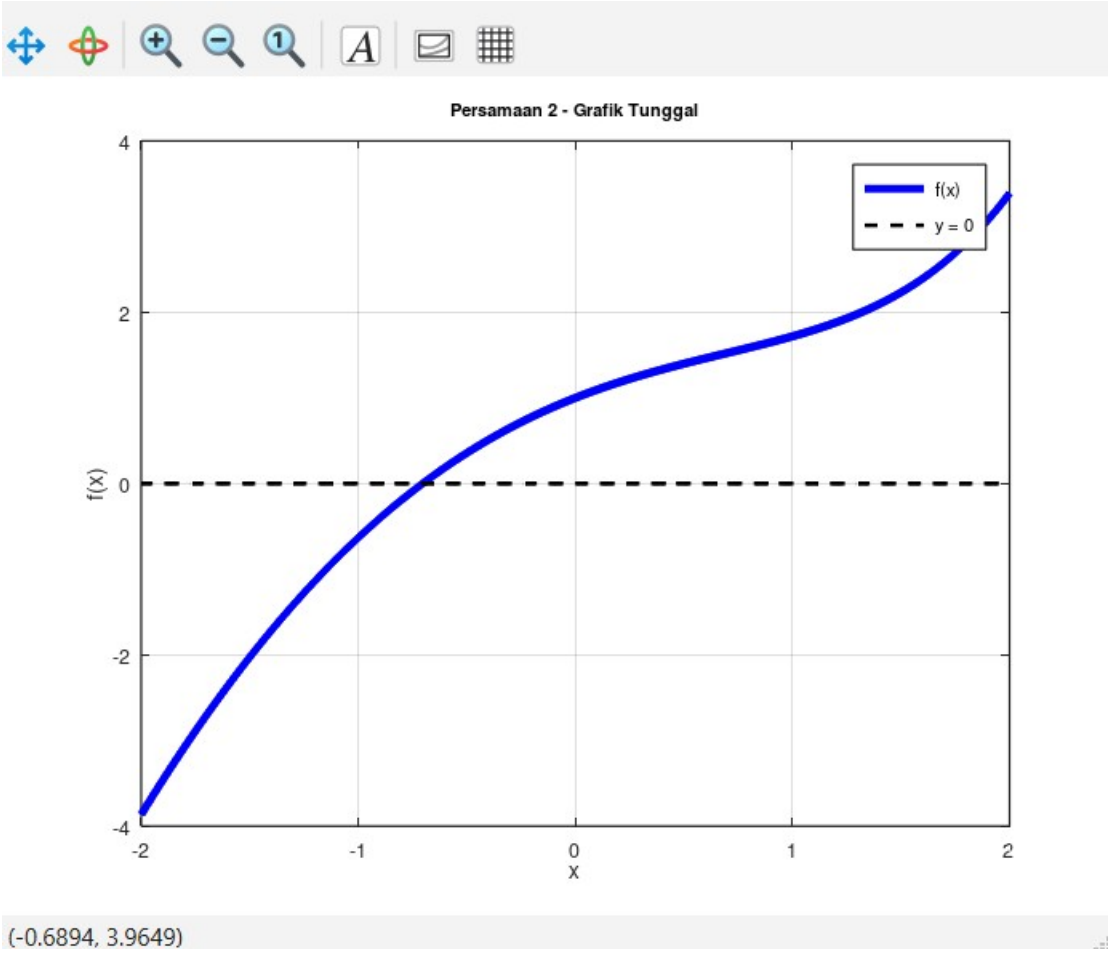
- Grafik tunggal: akar dicari dari $f(x) = 0$, dengan $f(x) = e^x - x^2$
- Grafik ganda: akar dicari dari $g_1(x) = g_2(x)$, dengan $g_1(x) = e^x$ dan $g_2(x) = x^2$

Penjelasan garis besar:

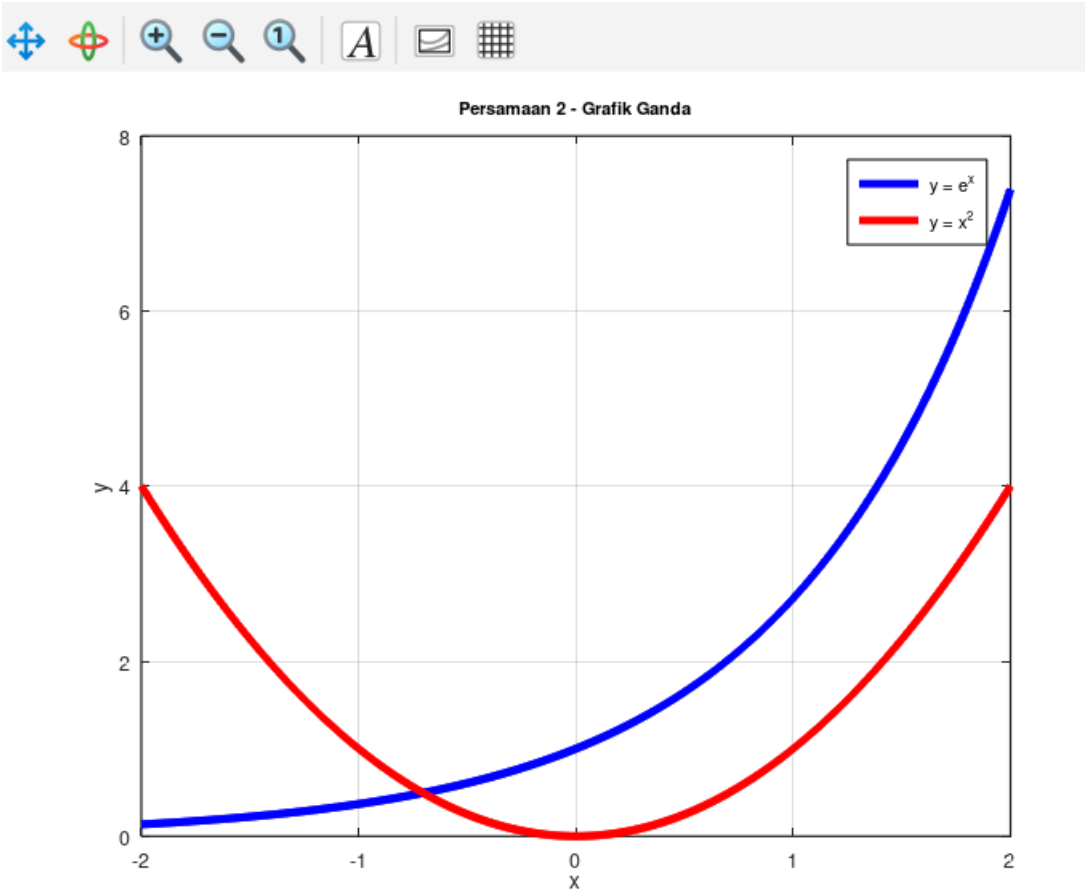
1. Bagian INPUT mendefinisikan fungsi f, g1, dan g2 sebagai anonymous function (@(x)). Operator .^ dipakai agar fungsi dapat menerima vektor x. Pada persamaan ini g2 berupa fungsi kuadrat x^2, sehingga tidak perlu memakai ones(size(x)) seperti pada konstanta.
2. Rentang sumbu x ditentukan oleh xmin = -2 dan xmax = 2. Variabel judul, ket1, dan ket2 mengatur judul dan legend grafik.
3. Perintah x = linspace(xmin, xmax, 1000) membentuk 1000 titik x berjarak sama pada selang $[-2, 2]$.
4. Figure 1 (grafik tunggal) menggambar f(x) sebagai garis biru tebal dan y = 0 sebagai garis putus-putus hitam. Akar adalah titik tempat kurva memotong garis y = 0.

5. Figure 2 (grafik ganda) menggambar $g_1(x) = e^x$ sebagai garis biru dan $g_2(x) = x^2$ sebagai garis merah. Akar adalah absis titik perpotongan kedua kurva.
6. Kedua grafik diberi grid on, label sumbu, judul, dan legend.

ii. Analisis hasil



Grafik tunggal: kurva $f(x) = e^x - x^2$ pada selang $[-2, 2]$ bernilai negatif di sisi kiri, yaitu $f(-2) \approx -3{,}865$ dan $f(-1) \approx -0{,}632$. Nilainya berubah tanda menjadi positif menjelang $x = 0$ ($f(0) = 1$). Kurva memotong sumbu x satu kali di sekitar $x \approx -0{,}70$. Rincian perubahan tandanya: $f(-0{,}71) \approx -0{,}013$ dan $f(-0{,}70) \approx +0{,}007$. Setelah titik itu kurva tetap di atas sumbu x, dengan $f(1) \approx 1{,}718$ dan $f(2) \approx 3{,}389$. Hanya ada satu akar karena turunannya, $f'(x) = e^x - 2x$, selalu positif. Untuk $x < 0$ kedua sukunya positif, sedangkan untuk $x \geq 0$ nilai minimumnya di $x = \ln 2$ adalah $f' = 2 - 2 \ln 2 \approx 0{,}61 > 0$. Jadi fungsi selalu naik.



(0.84055, 5.1439)

Grafik ganda: pada sisi kiri, kurva eksponensial $g_1(x) = e^x$ berada di bawah parabola $g_2(x) = x^2$ (di $x = -2$: $e^x \approx 0{,}135$ sedangkan $x^2 = 4$). Kedua kurva berpotongan di $x \approx -0{,}70$ pada ketinggian $y \approx 0{,}495$. Setelah titik itu e^x berada di atas x^2 (di $x = 0$: $1 > 0$, di $x = 2$: $7{,}389 > 4$) dan tidak berpotongan lagi. Hasil ini konsisten dengan grafik tunggal, karena $f(x) = g_1(x) - g_2(x)$. Titik potong dua kurva pada grafik ganda sama dengan titik potong $f(x)$ dengan sumbu x pada grafik tunggal.

Kesimpulan: kedua metode grafik memberikan akar yang sama, yaitu $x \approx -0{,}70$, dan akar ini satu-satunya pada selang $[-2, 2]$ (bahkan untuk semua x real karena f selalu naik). Nilai ini dapat dipakai sebagai selang awal $[-0{,}71; -0{,}70]$ atau tebakan awal pada metode numerik seperti bisection dan Newton-Raphson.

3. Analisis Source Code “GRAFIK GAMBAR PERSAMAAN 03”

i. Analisis source code

```
f  = @(x) log10(x+2) + 2*x - 3;
g1 = @(x) log10(x+2);
g2 = @(x) 3 - 2*x;

% Rentang sumbu x
xmin = -1.9;
xmax = 3;

% Judul dan keterangan
judul = 'Persamaan 3';
ket1  = 'y = log(x+2)';
ket2  = 'y = 3 - 2x';
% ======================================================================

x = linspace(xmin, xmax, 1000);

% ---------------------- GRAFIK TUNGGAL ----------------------
figure(1);
plot(x, f(x), 'b-', 'LineWidth', 2); hold on;
plot(x, zeros(size(x)), 'k--', 'LineWidth', 1);
grid on;
xlabel('x'); ylabel('f(x)');
title([judul ' - Grafik Tunggal']);
legend('f(x)', 'y = 0', 'Location', 'best');

% ---------------------- GRAFIK GANDA ------------------------
figure(2);
plot(x, g1(x), 'b-', 'LineWidth', 2); hold on;
plot(x, g2(x), 'r-', 'LineWidth', 2);
grid on;
xlabel('x'); ylabel('y');
title([judul ' - Grafik Ganda']);
legend(ket1, ket2, 'Location', 'best');
```

Program ini bertujuan menggambar kurva fungsi dengan metode grafik tunggal dan grafik ganda untuk memperkirakan letak akar persamaan nonlinear $\log_{10}(x+2) + 2x - 3 = 0$.

Rumus yang dihitung:

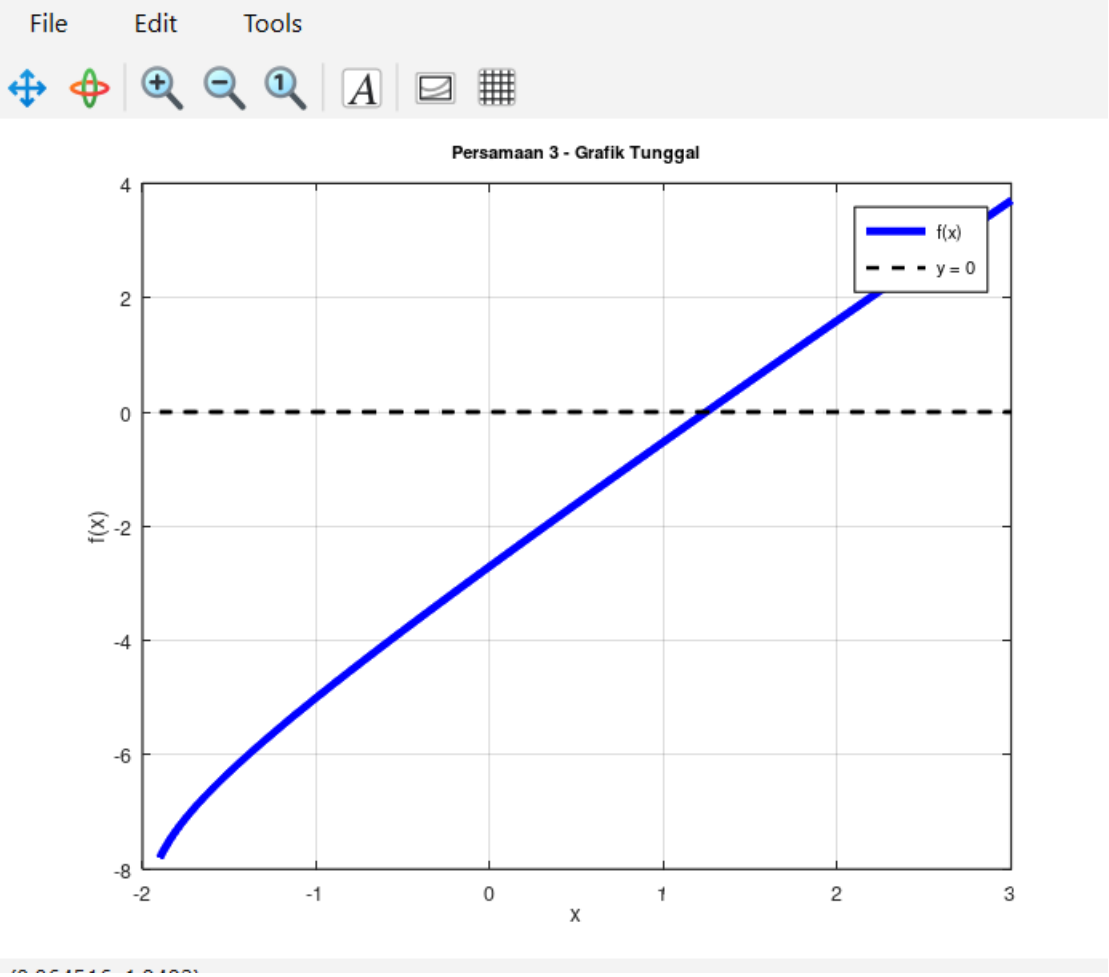
- Grafik tunggal: akar dicari dari $f(x) = 0$, dengan $f(x) = \log_{10}(x+2) + 2x - 3$
- Grafik ganda: akar dicari dari $g_1(x) = g_2(x)$, dengan $g_1(x) = \log_{10}(x+2)$ dan $g_2(x) = 3 - 2x$ (hasil memindahkan suku $2x - 3$ ke ruas kanan)

Penjelasan garis besar:

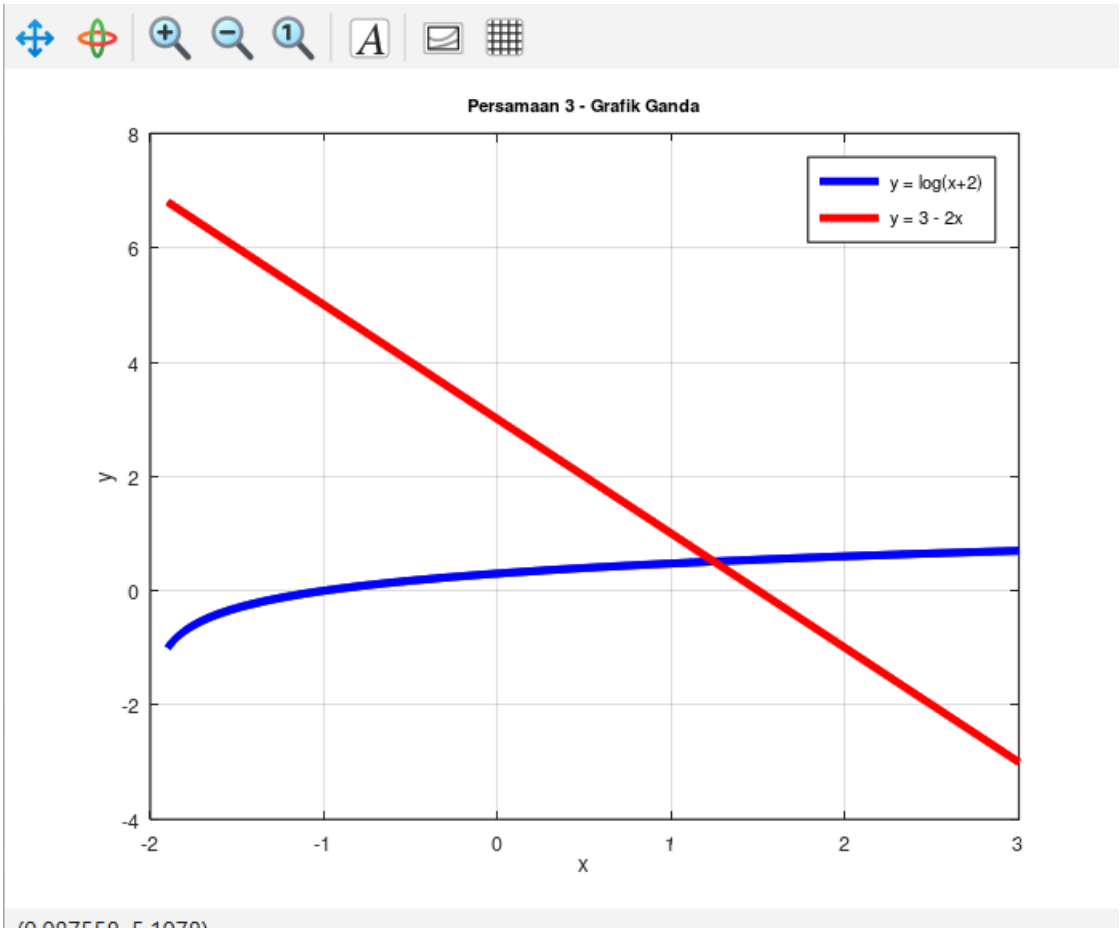
1. Bagian INPUT mendefinisikan fungsi f, g1, dan g2 sebagai anonymous function (@(x)). Fungsi logaritma ditulis dengan log, yaitu logaritma basis 10. Pada Octave, log berarti logaritma natural, jadi pemakaian log menentukan bahwa akar yang dicari adalah untuk logaritma basis 10.
2. Rentang sumbu x ditentukan oleh xmin = -1.9 dan xmax = 3. Batas bawah dipilih −1,9, bukan −2, karena $\log_{10}(x+2)$ hanya terdefinisi untuk $x + 2 > 0$, yaitu $x > -2$. Jika selang dimulai dari $x = -2$, fungsi bernilai $-\infty$ pada titik itu.
3. Perintah x = linspace(xmin, xmax, 1000) membentuk 1000 titik x berjarak sama pada selang [−1,9; 3].

4. Figure 1 (grafik tunggal) menggambar f(x) sebagai garis biru tebal dan y = 0 sebagai garis putus-putus hitam. Akar adalah titik tempat kurva memotong garis y = 0.
5. Figure 2 (grafik ganda) menggambar $g_1(x)$ sebagai garis biru dan $g_2(x)$ sebagai garis merah. Akar adalah absis titik perpotongan kedua kurva.
6. Kedua grafik diberi grid on, label sumbu, judul, dan legend.

ii. Analisis hasil



Grafik tunggal: kurva f(x) = log(x+2) + 2x − 3 pada selang [−1,9; 3] naik dari f(−1,9) = −7,8 menuju f(3) ≈ 3,699. Nilai fungsi negatif di sisi kiri dan tengah, yaitu f(−1) = −5, f(0) ≈ −2,699, dan f(1) ≈ −0,523. Nilainya berubah tanda menjadi positif di antara x = 1 dan x = 2 (f(2) ≈ 1,602). Kurva memotong sumbu x di sekitar x ≈ 1,244. Rincian perubahan tandanya: f(1,24) ≈ −0,009 dan f(1,25) ≈ +0,012. Hanya ada satu titik potong karena turunannya, f′(x) = 1/((x+2) ln 10) + 2, selalu positif untuk x > −2, sehingga fungsi selalu naik. Pada selang ini kurva juga melengkung cukup tajam di dekat x = −1,9 karena sifat fungsi logaritma yang turun curam mendekati x = −2.



Grafik ganda: kurva logaritma $g_1(x) = \log(x+2)$ naik perlahan dari -1 di $x = -1{,}9$ sampai sekitar 0,699 di $x = 3$. Garis lurus $g_2(x) = 3 - 2x$ turun dari 6,8 sampai -3. Di sisi kiri garis berada di atas kurva logaritma (di $x = 0$: $g_2 = 3$ sedangkan $g_1 \approx 0{,}301$). Kedua kurva berpotongan di $x \approx 1{,}244$ pada ketinggian $y \approx 0{,}511$. Setelah titik itu garis berada di bawah kurva logaritma (di $x = 2$: $g_2 = -1$ sedangkan $g_1 \approx 0{,}602$) dan tidak berpotongan lagi. Hasil ini konsisten dengan grafik tunggal, karena $f(x) = g_1(x) - g_2(x)$. Titik potong dua kurva pada grafik ganda sama dengan titik potong $f(x)$ dengan sumbu x pada grafik tunggal.

Kesimpulan: kedua metode grafik memberikan akar yang sama, yaitu $x \approx 1{,}244$, dan akar ini satu-satunya pada selang $[-1{,}9; 3]$. Nilai ini dapat dipakai sebagai selang awal $[1{,}24; 1{,}25]$ atau tebakan awal pada metode numerik seperti bisection dan Newton-Raphson.

4.Analisis Source Code “metode BISEKSI persamaan 01”

i. Analisis source code

```
switch pilih
  case 1   % y = 3x^3 - 5x^2 + 4x - 7 dengan y = 1
    f = @(x) 3*x.^3 - 5*x.^2 + 4*x - 7 - 1;
    a = 1;   b = 2;
  case 2   % y = e^x - x^2 dengan y = 0
    f = @(x) exp(x) - x.^2;
    a = -1;  b = 0;
  case 3   % y = log(x+2) + 2x - 3 dengan y = 0 (log basis 10)
    f = @(x) log10(x+2) + 2*x - 3;
    a = 1;   b = 2;
end
% ======================================================================

% ------------------------- PROSES BISEKSI ----------------------------
fa = f(a);
fb = f(b);

if fa*fb > 0
  error('Warning: f(a) dan f(b) sama tanda. Ganti interval a dan b.');
end

fprintf('\n=== METODE BISEKSI | Persamaan %d | interval [%g, %g] ===\n', pilih, a, b);
fprintf('Iter\ta\t\tb\t\tfa\t\tfb\t\tc\t\tfc\n');

for i = 1:n
  c  = (a+b)/2;          % titik tengah
  fc = f(c);
  fprintf('%d\t%f\t%f\t%f\t%f\t%f\t%f\n', i, a, b, fa, fb, c, fc);

  % berhenti jika error mencapai toleransi
  if abs(c-a) <= tol || abs(c-b) <= tol
    break;
  end

  if fa*fc == 0          % akar tepat ditemukan
    break;
  elseif fa*fc < 0       % akar di antara a dan c
    b = c;  fb = f(b);
  else                   % akar di antara c dan b
    a = c;  fa = f(a);
  end
end
```

Program ini bertujuan mencari akar persamaan nonlinear f(x) = 0 pada selang [a, b] dengan membagi dua selang secara berulang sampai galatnya tidak lebih dari toleransi.

Rumus yang dihitung:

- Titik tengah: c = (a + b) / 2
- Jika f(a)·f(c) < 0, akar berada di [a, c], sehingga b = c
- Jika f(a)·f(c) > 0, akar berada di [c, b], sehingga a = c
- Syarat berhenti: |c − a| ≤ tol atau |c − b| ≤ tol
- Untuk Persamaan 1: f(x) = $3x^3 - 5x^2 + 4x - 7 - 1 = 3x^3 - 5x^2 + 4x - 8$

Penjelasan garis besar:

1. Bagian INPUT berisi pilih (nomor persamaan), n (maksimal iterasi, 20), dan tol (toleransi, 0,0001). Perintah switch memilih fungsi f dan selang awal [a, b] sesuai nilai pilih. Pada pilih = 1 dipakai f(x) = $3x^3 - 5x^2 + 4x - 8$ dengan selang [1, 2]. Konstanta y = 1 sudah dipindahkan ke ruas kiri agar bentuknya f(x) = 0.
2. Program menghitung fa = f(a) dan fb = f(b), lalu memeriksa syarat fa*fb > 0. Jika kedua nilai bertanda sama, tidak dijamin ada akar di selang itu, sehingga program berhenti dengan pesan error.
3. Program mencetak judul dan header tabel dengan fprintf.

4. Di dalam perulangan for i = 1:n, setiap iterasi menghitung titik tengah c dan nilai fc = f(c), lalu mencetak satu baris tabel (a, b, fa, fb, c, fc).
5. Syarat berhenti diperiksa setelah baris dicetak. Jika |c − a| ≤ tol atau |c − b| ≤ tol, perulangan dihentikan dengan break.
6. Jika belum berhenti, selang dipersempit. Bila fa*fc == 0, akar tepat ditemukan dan perulangan berhenti. Bila fa*fc < 0, nilai b diganti c. Selain itu, nilai a diganti c. Nilai fa atau fb diperbarui sesuai batas yang berubah.
7. Setelah perulangan, program mencetak akar hampiran (c) dan jumlah iterasi (i).

ii. Analisis hasil

```
=== METODE BISEKSI | Persamaan 1 | interval [1, 2] ===
Iter    a               b               fa              fb              c               fc
1       1.000000        2.000000        -6.000000       4.000000        1.500000        -3.125000
2       1.500000        2.000000        -3.125000       4.000000        1.750000        -0.234375
3       1.750000        2.000000        -0.234375       4.000000        1.875000        1.697266
4       1.750000        1.875000        -0.234375       1.697266        1.812500        0.687256
5       1.750000        1.812500        -0.234375       0.687256        1.781250        0.215668
6       1.750000        1.781250        -0.234375       0.215668        1.765625        -0.012012
7       1.765625        1.781250        -0.012012       0.215668        1.773438        0.101159
8       1.765625        1.773438        -0.012012       0.101159        1.769531        0.044406
9       1.765625        1.769531        -0.012012       0.044406        1.767578        0.016155
10      1.765625        1.767578        -0.012012       0.016155        1.766602        0.002061
11      1.765625        1.766602        -0.012012       0.002061        1.766113        -0.004978
12      1.766113        1.766602        -0.004978       0.002061        1.766357        -0.001459
13      1.766357        1.766602        -0.001459       0.002061        1.766479        0.000301
14      1.766357        1.766479        -0.001459       0.000301        1.766418        -0.000579

Akar hampiran = 1.766418
Jumlah iterasi = 14
octave:13>
```

Program dijalankan dengan pilih = 1, n = 20, dan tol = 0,0001, pada selang awal [1, 2]. Syarat awal terpenuhi karena f(1) = −6 dan f(2) = 4 berbeda tanda, sehingga ada akar di selang tersebut.

Pada iterasi 1, titik tengah c = 1,5 menghasilkan f(c) = −3,125 (negatif). Akar berada di sisi kanan, sehingga a diganti c. Pada iterasi 2, c = 1,75 menghasilkan f(c) = −0,234375 (negatif), sehingga a diganti lagi. Pada iterasi 3, c = 1,875 menghasilkan f(c) = 1,697266 (positif), sehingga b diganti c dan selang menjadi [1,75; 1,875]. Selang terus menyempit, dan nilai | f(c)| semakin kecil: 0,687256 (iterasi 4), 0,215668 (iterasi 5), 0,012012 (iterasi 6), hingga 0,000579 pada iterasi 14. Tanda f(c) bergantian antara negatif dan positif, tanda bahwa titik tengah bergerak mengapit akar dari kedua sisi.

Pada iterasi 14, selang berada di [1,766357; 1,766479] dengan c = 1,766418 dan f(c) = −0,000579. Pada titik ini |c − a| = 0,000061 sudah di bawah toleransi 0,0001, sehingga perulangan berhenti. Pada iterasi 13, |c − a| = 0,000122 masih di atas toleransi, jadi iterasi ke-14 memang yang pertama memenuhi syarat. Hasil ini sesuai dengan teori, karena galat pada iterasi ke-i adalah $(b_0 - a_0)/2^i = 1/2^i$. Syarat $1/2^i \leq 0{,}0001$ menghasilkan $i \geq 13{,}29$, sehingga dibutuhkan 14 iterasi.

Hasil akhirnya adalah akar hampiran x ≈ 1,766418 setelah 14 iterasi, jauh di bawah batas maksimal 20 iterasi. Nilai ini sesuai dengan perkiraan metode grafik pada Bagian 1 (x ≈ 1,77). Karena f(x) selalu naik, akar ini satu-satunya pada selang tersebut.

5.Analisis Source Code "metode BISEKSI persamaan 02"

i. Analisis source code

```
switch pilih
  case 1   % y = 3x^3 - 5x^2 + 4x - 7 dengan y = 1
    f = @(x) 3*x.^3 - 5*x.^2 + 4*x - 7 - 1;
    a = 1;   b = 2;
  case 2   % y = e^x - x^2 dengan y = 0
    f = @(x) exp(x) - x.^2;
    a = -1;  b = 0;
  case 3   % y = log(x+2) + 2x - 3 dengan y = 0 (log basis 10)
    f = @(x) log10(x+2) + 2*x - 3;
    a = 1;   b = 2;
end
% =========================================================================

% ------------------------- PROSES BISEKSI ----------------------------
fa = f(a);
fb = f(b);

if fa*fb > 0
  error('Warning: f(a) dan f(b) sama tanda. Ganti interval a dan b.');
end

fprintf('\n=== METODE BISEKSI | Persamaan %d | interval [%g, %g] ===\n', pilih, a, b);
fprintf('Iter\ta\t\tb\t\tfa\t\tfb\t\tc\t\tfc\n');

for i = 1:n
  c  = (a+b)/2;          % titik tengah
  fc = f(c);
  fprintf('%d\t%f\t%f\t%f\t%f\t%f\t%f\n', i, a, b, fa, fb, c, fc);

  % berhenti jika error mencapai toleransi
  if abs(c-a) <= tol || abs(c-b) <= tol
    break;
  end

  if fa*fc == 0          % akar tepat ditemukan
    break;
  elseif fa*fc < 0       % akar di antara a dan c
    b = c;  fb = f(b);
  else                   % akar di antara c dan b
    a = c;  fa = f(a);
  end
end
```

Program ini bertujuan mencari akar persamaan nonlinear f(x) = 0 pada selang [a, b] dengan membagi dua selang secara berulang sampai galatnya tidak lebih dari toleransi.

Rumus yang dihitung:

- Titik tengah: c = (a + b) / 2
- Jika f(a)·f(c) < 0, akar berada di [a, c], sehingga b = c
- Jika f(a)·f(c) > 0, akar berada di [c, b], sehingga a = c
- Syarat berhenti: |c − a| ≤ tol atau |c − b| ≤ tol
- Untuk Persamaan 2: f(x) = $e^x - x^2$

Penjelasan garis besar:

1. Bagian INPUT berisi pilih (nomor persamaan), n (maksimal iterasi, 20), dan tol (toleransi, 0,0001). Perintah switch memilih fungsi f dan selang awal [a, b] sesuai nilai pilih. Pada pilih = 2 dipakai f(x) = $e^x - x^2$ dengan selang [−1, 0].

2. Program menghitung fa = f(a) dan fb = f(b), lalu memeriksa syarat fa*fb > 0. Jika kedua nilai bertanda sama, program berhenti dengan pesan error karena akar tidak dijamin ada di selang itu.
3. Program mencetak judul dan header tabel dengan fprintf.
4. Di dalam perulangan for i = 1:n, setiap iterasi menghitung titik tengah c dan nilai fc = f(c), lalu mencetak satu baris tabel (a, b, fa, fb, c, fc).
5. Syarat berhenti diperiksa setelah baris dicetak. Jika |c − a| ≤ tol atau |c − b| ≤ tol, perulangan dihentikan dengan break.
6. Jika belum berhenti, selang dipersempit. Bila fa*fc == 0, akar tepat ditemukan dan perulangan berhenti. Bila fa*fc < 0, nilai b diganti c. Selain itu, nilai a diganti c. Nilai fa atau fb diperbarui sesuai batas yang berubah.
7. Setelah perulangan, program mencetak akar hampiran (c) dan jumlah iterasi (i).

ii. Analisis hasil

```
=== METODE BISEKSI | Persamaan 2 | interval [-1, 0] ===
Iter    a               b               fa              fb              c               fc
1       -1.000000       0.000000        -0.632121       1.000000        -0.500000       0.356531

2       -1.000000       -0.500000       -0.632121       0.356531        -0.750000       -0.09013
3
3       -0.750000       -0.500000       -0.090133       0.356531        -0.625000       0.144636

4       -0.750000       -0.625000       -0.090133       0.144636        -0.687500       0.030175

5       -0.750000       -0.687500       -0.090133       0.030175        -0.718750       -0.02924
0
6       -0.718750       -0.687500       -0.029240       0.030175        -0.703125       0.000651

7       -0.718750       -0.703125       -0.029240       0.000651        -0.710938       -0.01424
9
8       -0.710938       -0.703125       -0.014249       0.000651        -0.707031       -0.00678
7
9       -0.707031       -0.703125       -0.006787       0.000651        -0.705078       -0.00306
5
10      -0.705078       -0.703125       -0.003065       0.000651        -0.704102       -0.00120
6
11      -0.704102       -0.703125       -0.001206       0.000651        -0.703613       -0.00027
7
12      -0.703613       -0.703125       -0.000277       0.000651        -0.703369       0.000187

13      -0.703613       -0.703369       -0.000277       0.000187        -0.703491       -0.00004
5
14      -0.703491       -0.703369       -0.000045       0.000187        -0.703430       0.000071


Akar hampiran = -0.703430
Jumlah iterasi = 14
octave:14>
```

Program dijalankan dengan pilih = 2, n = 20, dan tol = 0,0001, pada selang awal [−1, 0]. Syarat awal terpenuhi karena f(−1) = −0,632121 dan f(0) = 1 berbeda tanda, sehingga ada akar di selang tersebut.

Pada iterasi 1, titik tengah c = −0,5 menghasilkan f(c) = 0,356531 (positif), sehingga b diganti c dan selang menjadi [−1; −0,5]. Pada iterasi 2, c = −0,75 menghasilkan f(c) = −0,090133 (negatif), sehingga a diganti c dan selang menjadi [−0,75; −0,5]. Pada iterasi 3, c = −0,625 menghasilkan f(c) = 0,144636 (positif), sehingga b diganti c. Selang terus menyempit dan tanda f(c) bergantian antara positif dan negatif, tanda bahwa titik tengah mengapit akar dari kedua sisi. Pada iterasi 6, c = −0,703125 menghasilkan f(c) = 0,000651, sangat dekat dengan nol. Setelah itu nilai |f(c)| bergerak naik turun di sekitar nol hingga mencapai 0,000071 pada iterasi 14.

Pada iterasi 14, selang berada di [−0,703491; −0,703369] dengan c = −0,703430 dan f(c) = 0,000071. Pada titik ini |c − a| = 0,000061 sudah di bawah toleransi 0,0001, sehingga

perulangan berhenti. Pada iterasi 13, |c − a| = 0,000122 masih di atas toleransi, jadi iterasi ke-14 adalah yang pertama memenuhi syarat. Hasil ini sesuai dengan teori, karena galat pada iterasi ke-i adalah $(b_0 - a_0)/2^i = 1/2^i$. Syarat $1/2^i \leq 0{,}0001$ menghasilkan $i \geq 13{,}29$, sehingga dibutuhkan 14 iterasi.

Hasil akhirnya adalah akar hampiran $x \approx -0{,}703430$ setelah 14 iterasi, di bawah batas maksimal 20 iterasi. Nilai ini sesuai dengan perkiraan metode grafik pada Bagian 2 ($x \approx -0{,}70$). Karena f(x) selalu naik, akar ini satu-satunya pada selang tersebut.

Catatan tampilan: beberapa nilai fc pada output terpotong ke baris berikutnya (misalnya −0,09013 / 3) karena lebar jendela command window Octave, bukan kesalahan program. Karakter aneh di awal output ("叒🈵") hanyalah sisa karakter terminal dan tidak memengaruhi hasil.

6.Analisis Source Code “metode BISEKSI persamaan 03”

i. Analisis source code

```
switch pilih
  case 1   % y = 3x^3 - 5x^2 + 4x - 7 dengan y = 1
    f = @(x) 3*x.^3 - 5*x.^2 + 4*x - 7 - 1;
    a = 1;   b = 2;
  case 2   % y = e^x - x^2 dengan y = 0
    f = @(x) exp(x) - x.^2;
    a = -1;  b = 0;
  case 3   % y = log(x+2) + 2x - 3 dengan y = 0 (log basis 10)
    f = @(x) log10(x+2) + 2*x - 3;
    a = 1;   b = 2;
end
% =========================================================================

% ------------------------- PROSES BISEKSI -----------------------------
fa = f(a);
fb = f(b);

if fa*fb > 0
  error('Warning: f(a) dan f(b) sama tanda. Ganti interval a dan b.');
end

fprintf('\n=== METODE BISEKSI | Persamaan %d | interval [%g, %g] ===\n', pilih, a, b);
fprintf('Iter\ta\t\tb\t\tfa\t\tfb\t\tc\t\tfc\n');

for i = 1:n
  c  = (a+b)/2;          % titik tengah
  fc = f(c);
  fprintf('%d\t%f\t%f\t%f\t%f\t%f\t%f\n', i, a, b, fa, fb, c, fc);

  % berhenti jika error mencapai toleransi
  if abs(c-a) <= tol || abs(c-b) <= tol
    break;
  end

  if fa*fc == 0          % akar tepat ditemukan
    break;
  elseif fa*fc < 0       % akar di antara a dan c
    b = c;  fb = f(b);
  else                   % akar di antara c dan b
    a = c;  fa = f(a);
  end
end
```

Program ini bertujuan mencari akar persamaan nonlinear f(x) = 0 pada selang [a, b] dengan membagi dua selang secara berulang sampai galatnya tidak lebih dari toleransi.

Rumus yang dihitung:

- Titik tengah: c = (a + b) / 2
- Jika f(a)·f(c) < 0, akar berada di [a, c], sehingga b = c
- Jika f(a)·f(c) > 0, akar berada di [c, b], sehingga a = c

- Syarat berhenti: $|c - a| \leq tol$ atau $|c - b| \leq tol$
- Untuk Persamaan 3: $f(x) = \log_{10}(x + 2) + 2x - 3$

Penjelasan garis besar:

1. Bagian INPUT berisi pilih (nomor persamaan), n (maksimal iterasi, 20), dan tol (toleransi, 0,0001). Perintah switch memilih fungsi f dan selang awal [a, b] sesuai nilai pilih. Pada pilih = 3 dipakai $f(x) = \log_{10}(x + 2) + 2x - 3$ dengan selang [1, 2]. Fungsi log10 menandakan logaritma basis 10.
2. Program menghitung fa = f(a) dan fb = f(b), lalu memeriksa syarat fa*fb > 0. Jika kedua nilai bertanda sama, program berhenti dengan pesan error karena akar tidak dijamin ada di selang itu.
3. Program mencetak judul dan header tabel dengan fprintf.
4. Di dalam perulangan for i = 1:n, setiap iterasi menghitung titik tengah c dan nilai fc = f(c), lalu mencetak satu baris tabel (a, b, fa, fb, c, fc).
5. Syarat berhenti diperiksa setelah baris dicetak. Jika $|c - a| \leq tol$ atau $|c - b| \leq tol$, perulangan dihentikan dengan break.
6. Jika belum berhenti, selang dipersempit. Bila fa*fc == 0, akar tepat ditemukan dan perulangan berhenti. Bila fa*fc < 0, nilai b diganti c. Selain itu, nilai a diganti c. Nilai fa atau fb diperbarui sesuai batas yang berubah.
7. Setelah perulangan, program mencetak akar hampiran (c) dan jumlah iterasi (i).

ii. Analisis hasil

```
=== METODE BISEKSI | Persamaan 3 | interval [1, 2] ===
Iter    a               b               fa              fb              c               fc
1       1.000000        2.000000        -0.522879       1.602060        1.500000        0.544068

2       1.000000        1.500000        -0.522879       0.544068        1.250000        0.011883

3       1.000000        1.250000        -0.522879       0.011883        1.125000        -0.25515
0
4       1.125000        1.250000        -0.255150       0.011883        1.187500        -0.12155
0
5       1.187500        1.250000        -0.121550       0.011883        1.218750        -0.05481
3
6       1.218750        1.250000        -0.054813       0.011883        1.234375        -0.02146
0
7       1.234375        1.250000        -0.021460       0.011883        1.242188        -0.00478
7
8       1.242188        1.250000        -0.004787       0.011883        1.246094        0.003549

9       1.242188        1.246094        -0.004787       0.003549        1.244141        -0.00061
9
10      1.244141        1.246094        -0.000619       0.003549        1.245117        0.001465

11      1.244141        1.245117        -0.000619       0.001465        1.244629        0.000423

12      1.244141        1.244629        -0.000619       0.000423        1.244385        -0.00009
8
13      1.244385        1.244629        -0.000098       0.000423        1.244507        0.000162

14      1.244385        1.244507        -0.000098       0.000162        1.244446        0.000032


Akar hampiran = 1.244446
Jumlah iterasi = 14
octave:15>
```

Program dijalankan dengan pilih = 3, n = 20, dan tol = 0,0001, pada selang awal [1, 2]. Syarat awal terpenuhi karena f(1) = −0,522879 dan f(2) = 1,602060 berbeda tanda, sehingga ada akar di selang tersebut.

Pada iterasi 1, titik tengah c = 1,5 menghasilkan f(c) = 0,544068 (positif), sehingga b diganti c dan selang menjadi [1; 1,5]. Pada iterasi 2, c = 1,25 menghasilkan f(c) = 0,011883 (positif), sehingga b diganti lagi dan selang menjadi [1; 1,25]. Nilai f(c) di sini sudah sangat dekat dengan nol, tetapi akar sebenarnya berada sedikit di sebelah kiri 1,25. Pada iterasi 3, c = 1,125 menghasilkan f(c) = −0,255150 (negatif), sehingga a diganti c dan selang menjadi [1,125; 1,25]. Selanjutnya selang terus menyempit dan tanda f(c) bergantian antara negatif dan positif (misalnya iterasi 8 positif, iterasi 9 negatif, iterasi 10 positif), tanda bahwa titik tengah mengapit akar dari kedua sisi. Nilai |f(c)| mengecil dari 0,544068 pada iterasi 1 menjadi 0,000032 pada iterasi 14.

Pada iterasi 14, selang berada di [1,244385; 1,244507] dengan c = 1,244446 dan f(c) = 0,000032. Pada titik ini |c − a| = 0,000061 sudah di bawah toleransi 0,0001, sehingga perulangan berhenti. Pada iterasi 13, |c − a| = 0,000122 masih di atas toleransi, jadi iterasi ke-14 adalah yang pertama memenuhi syarat. Hasil ini sesuai dengan teori, karena galat pada iterasi ke-i adalah $(b_0 - a_0)/2^i = 1/2^i$. Syarat $1/2^i \leq 0{,}0001$ menghasilkan $i \geq 13{,}29$, sehingga dibutuhkan 14 iterasi.

Hasil akhirnya adalah akar hampiran x ≈ 1,244446 setelah 14 iterasi, di bawah batas maksimal 20 iterasi. Nilai ini sesuai dengan perkiraan metode grafik pada Bagian 3 (x ≈ 1,244). Karena f(x) selalu naik, akar ini satu-satunya pada selang tersebut.

7.Analisis Source Code “metode REGULA FALSI persamaan 01”

```
switch pilih
  case 1   % y = 3x^3 - 5x^2 + 4x - 7 dengan y = 1
    f = @(x) 3*x.^3 - 5*x.^2 + 4*x - 7 - 1;
    a = 1;   b = 2;
  case 2   % y = e^x - x^2 dengan y = 0
    f = @(x) exp(x) - x.^2;
    a = -1;  b = 0;
  case 3   % y = log(x+2) + 2x - 3 dengan y = 0 (log basis 10)
    f = @(x) log10(x+2) + 2*x - 3;
    a = 1;   b = 2;
end
% =========================================================================

% ----------------------- PROSES REGULA-FALSI -------------------------
fa = f(a);
fb = f(b);

if fa*fb > 0
  error('Warning: f(a) dan f(b) sama tanda. Ganti interval a dan b.');
end

fprintf('\n=== METODE REGULA-FALSI | Persamaan %d | interval [%g, %g] ===\n', pilih, a, b);
fprintf('Iter\ta\t\tb\t\tfa\t\tfb\t\tc\t\tfc\n');

for i = 1:n
  c  = (a*fb - b*fa)/(fb - fa);   % rumus regula-falsi
  fc = f(c);
  fprintf('%d\t%f\t%f\t%f\t%f\t%f\t%f\n', i, a, b, fa, fb, c, fc);

  % berhenti jika error mencapai toleransi
  if abs(c-a) <= tol || abs(c-b) <= tol
    break;
  end

  if fa*fc == 0          % akar tepat ditemukan
    break;
  elseif fa*fc < 0       % akar di antara a dan c
    b = c;  fb = f(b);
  else                   % akar di antara c dan b
    a = c;  fa = f(a);
  end
end
```

i. Analisis source code

```
=== METODE REGULA-FALSI | Persamaan 1 | interval [1, 2] ===
Iter    a               b               fa              fb              c               fc
1       1.000000        2.000000        -6.000000       4.000000        1.600000        -2.11200
0
2       1.600000        2.000000        -2.112000       4.000000        1.738220        -0.39854
6
3       1.738220        2.000000        -0.398546       4.000000        1.761939        -0.06494
0
4       1.761939        2.000000        -0.064940       4.000000        1.765743        -0.01032
0
5       1.765743        2.000000        -0.010320       4.000000        1.766345        -0.00163
3
6       1.766345        2.000000        -0.001633       4.000000        1.766441        -0.00025
8

Akar hampiran = 1.766441
Jumlah iterasi = 6
octave:16>
```

Program ini bertujuan mencari akar persamaan nonlinear f(x) = 0 pada selang [a, b] dengan menarik garis lurus (garis sekan) antara titik (a, f(a)) dan (b, f(b)). Titik potong garis itu dengan sumbu x dipakai sebagai hampiran akar, dan proses diulang sampai galatnya tidak lebih dari toleransi.

Rumus yang dihitung:

- Titik hampiran: $c = (a \cdot f(b) - b \cdot f(a)) / (f(b) - f(a))$
- Jika $f(a) \cdot f(c) < 0$, akar berada di [a, c], sehingga b = c
- Jika $f(a) \cdot f(c) > 0$, akar berada di [c, b], sehingga a = c
- Syarat berhenti: $|c - a| \le tol$ atau $|c - b| \le tol$
- Untuk Persamaan 1: $f(x) = 3x^3 - 5x^2 + 4x - 7 - 1 = 3x^3 - 5x^2 + 4x - 8$

Penjelasan garis besar:

1. Bagian INPUT berisi pilih (nomor persamaan), n (maksimal iterasi, 20), dan tol (toleransi, 0,0001). Perintah switch memilih fungsi f dan selang awal [a, b]. Pada pilih = 1 dipakai $f(x) = 3x^3 - 5x^2 + 4x - 8$ dengan selang [1, 2]. Konstanta y = 1 dipindahkan ke ruas kiri agar bentuknya f(x) = 0.
2. Program menghitung fa = f(a) dan fb = f(b), lalu memeriksa syarat fa*fb > 0. Jika kedua nilai bertanda sama, akar tidak dijamin ada di selang itu, sehingga program berhenti dengan pesan error.
3. Program mencetak judul dan header tabel dengan fprintf.
4. Di dalam perulangan for i = 1:n, setiap iterasi menghitung c dengan rumus regula-falsi dan nilai fc = f(c), lalu mencetak satu baris tabel (a, b, fa, fb, c, fc).
5. Syarat berhenti diperiksa setelah baris dicetak. Jika $|c - a| \le tol$ atau $|c - b| \le tol$, perulangan dihentikan dengan break.
6. Jika belum berhenti, selang dipersempit. Bila fa*fc == 0, akar tepat ditemukan dan perulangan berhenti. Bila fa*fc < 0, nilai b diganti c. Selain itu, nilai a diganti c. Nilai fa atau fb diperbarui sesuai batas yang berubah.
7. Setelah perulangan, program mencetak akar hampiran (c) dan jumlah iterasi (i).

Perbedaan dengan metode biseksi hanya pada baris penentuan c. Biseksi memakai titik tengah (a + b)/2, sedangkan regula-falsi memakai titik potong garis sekan dengan sumbu x, sehingga nilai f(a) dan f(b) ikut diperhitungkan.

ii. Analisis hasil

Program dijalankan dengan pilih = 1, n = 20, dan tol = 0,0001, pada selang awal [1, 2]. Syarat awal terpenuhi karena f(1) = −6 dan f(2) = 4 berbeda tanda, sehingga ada akar di selang tersebut.

Pada iterasi 1, c = (1·4 − 2·(−6))/(4 − (−6)) = 1,6 dengan f(c) = −2,112 (negatif), sehingga a diganti c. Pada iterasi 2, c = 1,738220 dengan f(c) = −0,398546 (negatif), sehingga a diganti lagi. Hal yang sama terjadi pada iterasi 3 sampai 5, dengan f(c) berturut-turut −0,064940; −0,010320; dan −0,001633. Pada semua iterasi f(c) bernilai negatif, sehingga hanya batas kiri a yang bergerak naik (1 → 1,6 → 1,738220 → 1,761939 → 1,765743 → 1,766345), sedangkan batas kanan b tetap 2 dan nilai fb tetap 4. Titik hampiran mendekati akar dari satu sisi saja (dari kiri), karena kurva f(x) cembung di sekitar akar.

Pada iterasi 6, selang berada di [1,766345; 2] dengan c = 1,766441 dan f(c) = −0,000258. Pada titik ini |c − a| = 0,000096 sudah di bawah toleransi 0,0001, sehingga perulangan berhenti. Pada iterasi 5, |c − a| = 0,000602 masih di atas toleransi, jadi iterasi ke-6 adalah yang pertama memenuhi syarat. Nilai |f(c)| mengecil dengan perbandingan sekitar 0,16 setiap iterasi (2,112; 0,3985; 0,0649; 0,0103; 0,00163; 0,000258), yang menunjukkan konvergensi stabil.

Hasil akhirnya adalah akar hampiran x ≈ 1,766441 setelah 6 iterasi. Nilai ini sesuai dengan perkiraan metode grafik (x ≈ 1,77) dan sangat dekat dengan hasil biseksi (x ≈ 1,766418 setelah 14 iterasi). Regula-falsi lebih cepat, yaitu 6 iterasi dibanding 14 iterasi, karena memanfaatkan nilai fungsi di kedua ujung selang, bukan hanya membagi dua selang.

8.Analisis Source Code “metode REGULA FALSI persamaan 02”

i. Analisis source code

```
switch pilih
  case 1   % y = 3x^3 - 5x^2 + 4x - 7 dengan y = 1
    f = @(x) 3*x.^3 - 5*x.^2 + 4*x - 7 - 1;
    a = 1;   b = 2;
  case 2   % y = e^x - x^2 dengan y = 0
    f = @(x) exp(x) - x.^2;
    a = -1;  b = 0;
  case 3   % y = log(x+2) + 2x - 3 dengan y = 0 (log basis 10)
    f = @(x) log10(x+2) + 2*x - 3;
    a = 1;   b = 2;
end
% =========================================================================

% ---------------------- PROSES REGULA-FALSI -------------------------
fa = f(a);
fb = f(b);

if fa*fb > 0
  error('Warning: f(a) dan f(b) sama tanda. Ganti interval a dan b.');
end

fprintf('\n=== METODE REGULA-FALSI | Persamaan %d | interval [%g, %g] ===\n', pilih, a, b);
fprintf('Iter\ta\t\tb\t\tfa\t\tfb\t\tc\t\tfc\n');

for i = 1:n
  c  = (a*fb - b*fa)/(fb - fa);   % rumus regula-falsi
  fc = f(c);
  fprintf('%d\t%f\t%f\t%f\t%f\t%f\t%f\n', i, a, b, fa, fb, c, fc);

  % berhenti jika error mencapai toleransi
  if abs(c-a) <= tol || abs(c-b) <= tol
    break;
  end

  if fa*fc == 0          % akar tepat ditemukan
    break;
  elseif fa*fc < 0       % akar di antara a dan c
    b = c;  fb = f(b);
  else                   % akar di antara c dan b
    a = c;  fa = f(a);
  end
```

Program ini bertujuan mencari akar persamaan nonlinear f(x) = 0 pada selang [a, b] dengan menarik garis lurus (garis sekan) antara titik (a, f(a)) dan (b, f(b)). Titik potong garis itu dengan sumbu x dipakai sebagai hampiran akar, dan proses diulang sampai galatnya tidak lebih dari toleransi.

Rumus yang dihitung:

- Titik hampiran: $c = (a \cdot f(b) - b \cdot f(a)) / (f(b) - f(a))$
- Jika $f(a) \cdot f(c) < 0$, akar berada di [a, c], sehingga b = c
- Jika $f(a) \cdot f(c) > 0$, akar berada di [c, b], sehingga a = c
- Syarat berhenti: $|c - a| \le tol$ atau $|c - b| \le tol$
- Untuk Persamaan 2: $f(x) = e^x - x^2$

Penjelasan garis besar:

1. Bagian INPUT berisi pilih (nomor persamaan), n (maksimal iterasi, 20), dan tol (toleransi, 0,0001). Perintah switch memilih fungsi f dan selang awal [a, b]. Pada pilih = 2 dipakai $f(x) = e^x - x^2$ dengan selang [−1, 0].
2. Program menghitung fa = f(a) dan fb = f(b), lalu memeriksa syarat fa*fb > 0. Jika kedua nilai bertanda sama, akar tidak dijamin ada di selang itu, sehingga program berhenti dengan pesan error.
3. Program mencetak judul dan header tabel dengan fprintf.
4. Di dalam perulangan for i = 1:n, setiap iterasi menghitung c dengan rumus regula-falsi dan nilai fc = f(c), lalu mencetak satu baris tabel (a, b, fa, fb, c, fc).
5. Syarat berhenti diperiksa setelah baris dicetak. Jika $|c - a| \le tol$ atau $|c - b| \le tol$, perulangan dihentikan dengan break.
6. Jika belum berhenti, selang dipersempit. Bila fa*fc == 0, akar tepat ditemukan dan perulangan berhenti. Bila fa*fc < 0, nilai b diganti c. Selain itu, nilai a diganti c. Nilai fa atau fb diperbarui sesuai batas yang berubah.

7. Setelah perulangan, program mencetak akar hampiran (c) dan jumlah iterasi (i).

ii. Analisis hasil

```
=== METODE REGULA-FALSI | Persamaan 2 | interval [-1, 0] ===
Iter    a               b               fa              fb              c               fc
1       -1.000000       0.000000        -0.632121       1.000000        -0.612700       0.166485

2       -1.000000       -0.612700       -0.632121       0.166485        -0.693440       0.018994

3       -1.000000       -0.693440       -0.632121       0.018994        -0.702383       0.002061

4       -1.000000       -0.702383       -0.632121       0.002061        -0.703350       0.000222

5       -1.000000       -0.703350       -0.632121       0.000222        -0.703455       0.000024

6       -1.000000       -0.703455       -0.632121       0.000024        -0.703466       0.000003


Akar hampiran = -0.703466
Jumlah iterasi = 6
```

Program dijalankan dengan pilih = 2, n = 20, dan tol = 0,0001, pada selang awal [−1, 0]. Syarat awal terpenuhi karena f(−1) = −0,632121 dan f(0) = 1 berbeda tanda, sehingga ada akar di selang tersebut.

Pada iterasi 1, c = ((−1)·1 − 0·(−0,632121))/(1 − (−0,632121)) = −0,612700 dengan f(c) = 0,166485 (positif), sehingga b diganti c. Pada iterasi 2 sampai 5, f(c) tetap positif, yaitu 0,018994; 0,002061; 0,000222; dan 0,000024, sehingga b terus diganti c. Akibatnya hanya batas kanan b yang bergerak turun (0 → −0,612700 → −0,693440 → −0,702383 → −0,703350 → −0,703455), sedangkan batas kiri a tetap −1 dan nilai fa tetap −0,632121. Titik hampiran mendekati akar dari satu sisi saja (dari kanan), karena kurva f(x) cekung di sekitar akar ($f''(x) = e^x - 2 < 0$ untuk $x < \ln 2$), sehingga garis sekan selalu memotong sumbu x di sebelah kanan akar.

Pada iterasi 6, selang berada di [−1; −0,703455] dengan c = −0,703466 dan f(c) = 0,000003. Pada titik ini |c − b| = 0,000011 sudah di bawah toleransi 0,0001, sehingga perulangan berhenti. Pada iterasi 5, |c − b| = 0,000105 masih sedikit di atas toleransi, jadi iterasi ke-6 adalah yang pertama memenuhi syarat. Nilai |f(c)| mengecil dengan perbandingan sekitar 0,11 setiap iterasi (0,166485; 0,018994; 0,002061; 0,000222; 0,000024; 0,000003), yang menunjukkan konvergensi yang stabil dan cepat.

Hasil akhirnya adalah akar hampiran x ≈ −0,703466 setelah 6 iterasi. Nilai ini sesuai dengan perkiraan metode grafik (x ≈ −0,70) dan sangat dekat dengan hasil biseksi (x ≈ −0,703430 setelah 14 iterasi). Regula-falsi lebih cepat, yaitu 6 iterasi dibanding 14 iterasi, karena memanfaatkan nilai fungsi di kedua ujung selang. Nilai f(c) akhirnya (0,000003) juga jauh lebih kecil daripada hasil biseksi (0,000071), sehingga hampirannya lebih akurat.

9.Analisis Source Code “metode REGULA FALSI persamaan 03”

i. Analisis source code

```
switch pilih
  case 1   % y = 3x^3 - 5x^2 + 4x - 7 dengan y = 1
    f = @(x) 3*x.^3 - 5*x.^2 + 4*x - 7 - 1;
    a = 1;   b = 2;
  case 2   % y = e^x - x^2 dengan y = 0
    f = @(x) exp(x) - x.^2;
    a = -1;  b = 0;
  case 3   % y = log(x+2) + 2x - 3 dengan y = 0 (log basis 10)
    f = @(x) log10(x+2) + 2*x - 3;
    a = 1;   b = 2;
end
% ========================================================================

% ---------------------- PROSES REGULA-FALSI -------------------------
fa = f(a);
fb = f(b);

if fa*fb > 0
  error('Warning: f(a) dan f(b) sama tanda. Ganti interval a dan b.');
end

fprintf('\n=== METODE REGULA-FALSI | Persamaan %d | interval [%g, %g] ===\n', pilih, a, b);
fprintf('Iter\ta\t\tb\t\tfa\t\tfb\t\tc\t\tfc\n');

for i = 1:n
  c  = (a*fb - b*fa)/(fb - fa);   % rumus regula-falsi
  fc = f(c);
  fprintf('%d\t%f\t%f\t%f\t%f\t%f\n', i, a, b, fa, fb, c, fc);

  % berhenti jika error mencapai toleransi
  if abs(c-a) <= tol || abs(c-b) <= tol
    break;
  end

  if fa*fc == 0          % akar tepat ditemukan
    break;
  elseif fa*fc < 0       % akar di antara a dan c
    b = c;  fb = f(b);
  else                   % akar di antara c dan b
    a = c;  fa = f(a);
  end
end
```

Program ini bertujuan mencari akar persamaan nonlinear f(x) = 0 pada selang [a, b] dengan menarik garis lurus (garis sekan) antara titik (a, f(a)) dan (b, f(b)). Titik potong garis itu dengan sumbu x dipakai sebagai hampiran akar, dan proses diulang sampai galatnya tidak lebih dari toleransi.

Rumus yang dihitung:

- Titik hampiran: $c = (a \cdot f(b) - b \cdot f(a)) / (f(b) - f(a))$
- Jika $f(a) \cdot f(c) < 0$, akar berada di [a, c], sehingga b = c
- Jika $f(a) \cdot f(c) > 0$, akar berada di [c, b], sehingga a = c
- Syarat berhenti: $|c - a| \le tol$ atau $|c - b| \le tol$
- Untuk Persamaan 3: $f(x) = \log_{10}(x + 2) + 2x - 3$

Penjelasan garis besar:

1. Bagian INPUT berisi pilih (nomor persamaan), n (maksimal iterasi, 20), dan tol (toleransi, 0,0001). Perintah switch memilih fungsi f dan selang awal [a, b]. Pada pilih = 3 dipakai $f(x) = \log_{10}(x + 2) + 2x - 3$ dengan selang [1, 2]. Fungsi log10 menandakan logaritma basis 10.

2. Program menghitung fa = f(a) dan fb = f(b), lalu memeriksa syarat fa*fb > 0. Jika kedua nilai bertanda sama, akar tidak dijamin ada di selang itu, sehingga program berhenti dengan pesan error.
3. Program mencetak judul dan header tabel dengan fprintf.
4. Di dalam perulangan for i = 1:n, setiap iterasi menghitung c dengan rumus regula-falsi dan nilai fc = f(c), lalu mencetak satu baris tabel (a, b, fa, fb, c, fc).
5. Syarat berhenti diperiksa setelah baris dicetak. Jika |c − a| ≤ tol atau |c − b| ≤ tol, perulangan dihentikan dengan break.
6. Jika belum berhenti, selang dipersempit. Bila fa*fc == 0, akar tepat ditemukan dan perulangan berhenti. Bila fa*fc < 0, nilai b diganti c. Selain itu, nilai a diganti c. Nilai fa atau fb diperbarui sesuai batas yang berubah.
7. Setelah perulangan, program mencetak akar hampiran (c) dan jumlah iterasi (i).

ii. Analisis hasil

```
=== METODE REGULA-FALSI | Persamaan 3 | interval [1, 2] ===
Iter    a               b               fa              fb              c               fc
1       1.000000        2.000000        -0.522879       1.602060        1.246068        0.003493

2       1.000000        1.246068        -0.522879       0.003493        1.244435        0.000009

3       1.000000        1.244435        -0.522879       0.000009        1.244431        0.000000


Akar hampiran = 1.244431
Jumlah iterasi = 3
```

Program dijalankan dengan pilih = 3, n = 20, dan tol = 0,0001, pada selang awal [1, 2]. Syarat awal terpenuhi karena f(1) = −0,522879 dan f(2) = 1,602060 berbeda tanda, sehingga ada akar di selang tersebut.

Pada iterasi 1, c = (1·1,602060 − 2·(−0,522879))/(1,602060 − (−0,522879)) = 1,246068 dengan f(c) = 0,003493 (positif), sehingga b diganti c dan selang menjadi [1; 1,246068]. Pada iterasi 2, c = 1,244435 dengan f(c) = 0,000009 (positif), sehingga b diganti lagi. Pada iterasi 3, c = 1,244431 dengan f(c) ≈ 0,000000. Pada semua iterasi f(c) bernilai positif, sehingga hanya batas kanan b yang bergerak turun (2 → 1,246068 → 1,244435), sedangkan batas kiri a tetap 1 dan nilai fa tetap −0,522879. Titik hampiran mendekati akar dari satu sisi saja (dari kanan), karena kurva f(x) cekung di sekitar akar ($f''(x) = -1/((x+2)^2 \ln 10) < 0$), sehingga garis sekan selalu memotong sumbu x di sebelah kanan akar.

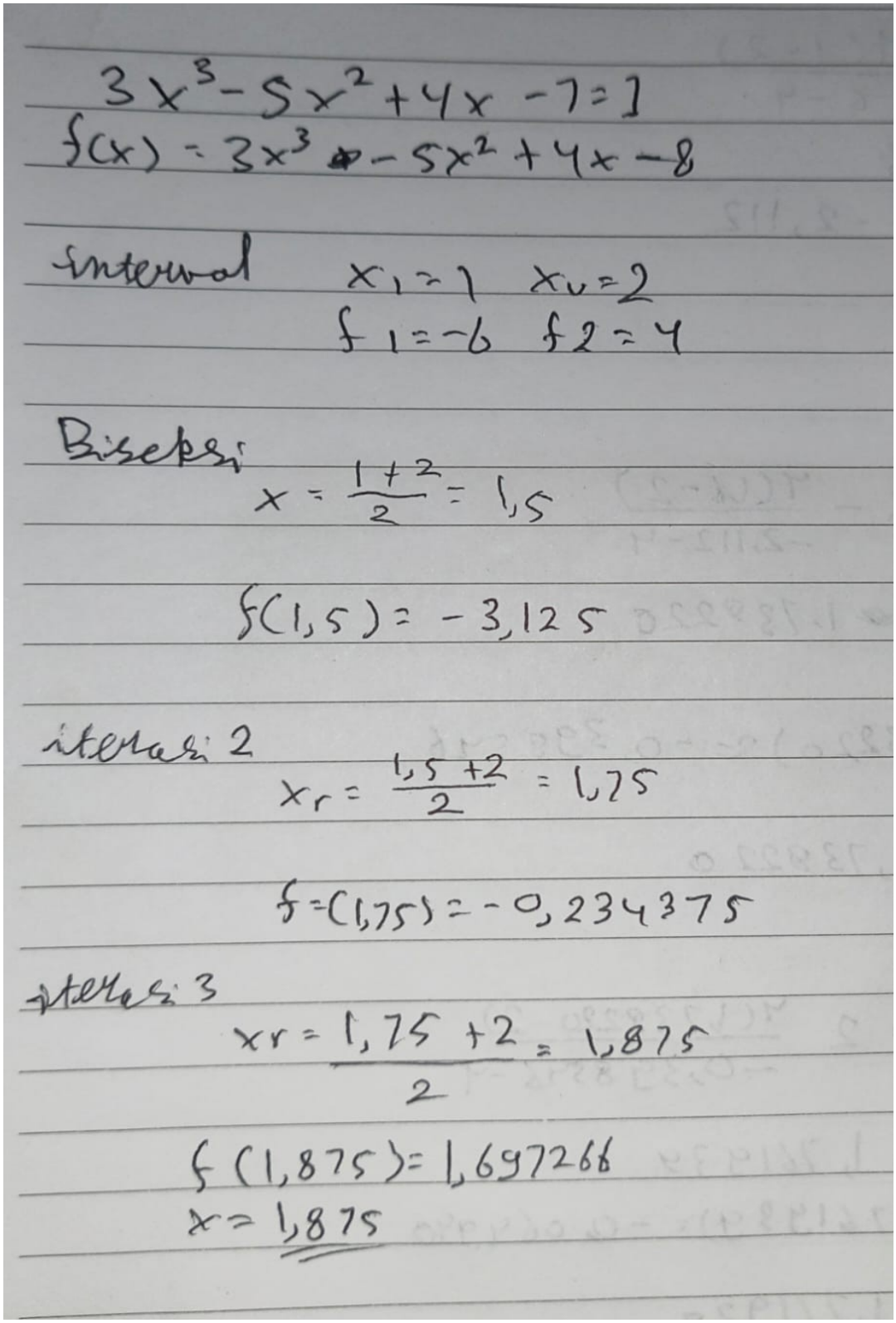
Pada iterasi 3, selang berada di [1; 1,244435] dengan c = 1,244431. Pada titik ini |c − b| = 0,000004 sudah di bawah toleransi 0,0001, sehingga perulangan berhenti. Pada iterasi 2, |c − b| = 0,001633 masih di atas toleransi, jadi iterasi ke-3 adalah yang pertama memenuhi syarat. Nilai |f(c)| mengecil sangat cepat (0,003493; 0,000009; 0,000000), yaitu sekitar 0,003 kali nilai sebelumnya pada setiap iterasi. Konvergensi ini jauh lebih cepat dibanding Persamaan 1 dan 2 karena kurva f(x) hampir lurus pada selang di dekat akar, sehingga garis sekan sangat mendekati kurva aslinya.

Hasil akhirnya adalah akar hampiran x ≈ 1,244431 setelah 3 iterasi. Nilai ini sesuai dengan perkiraan metode grafik (x ≈ 1,244) dan sangat dekat dengan hasil biseksi (x ≈ 1,244446 setelah 14 iterasi). Regula-falsi jauh lebih cepat, yaitu 3 iterasi dibanding 14 iterasi, karena memanfaatkan nilai fungsi di kedua ujung selang. Nilai f(c) akhirnya (≈ 0,000000) juga lebih kecil daripada hasil biseksi (0,000032), sehingga hampirannya lebih akurat.

BAB II

ANALISIS NUMERIK

1.BISEKSI PERSAMAAN 1



Iterasi 1

Iterasi 2

Iterasi 3

Hasil

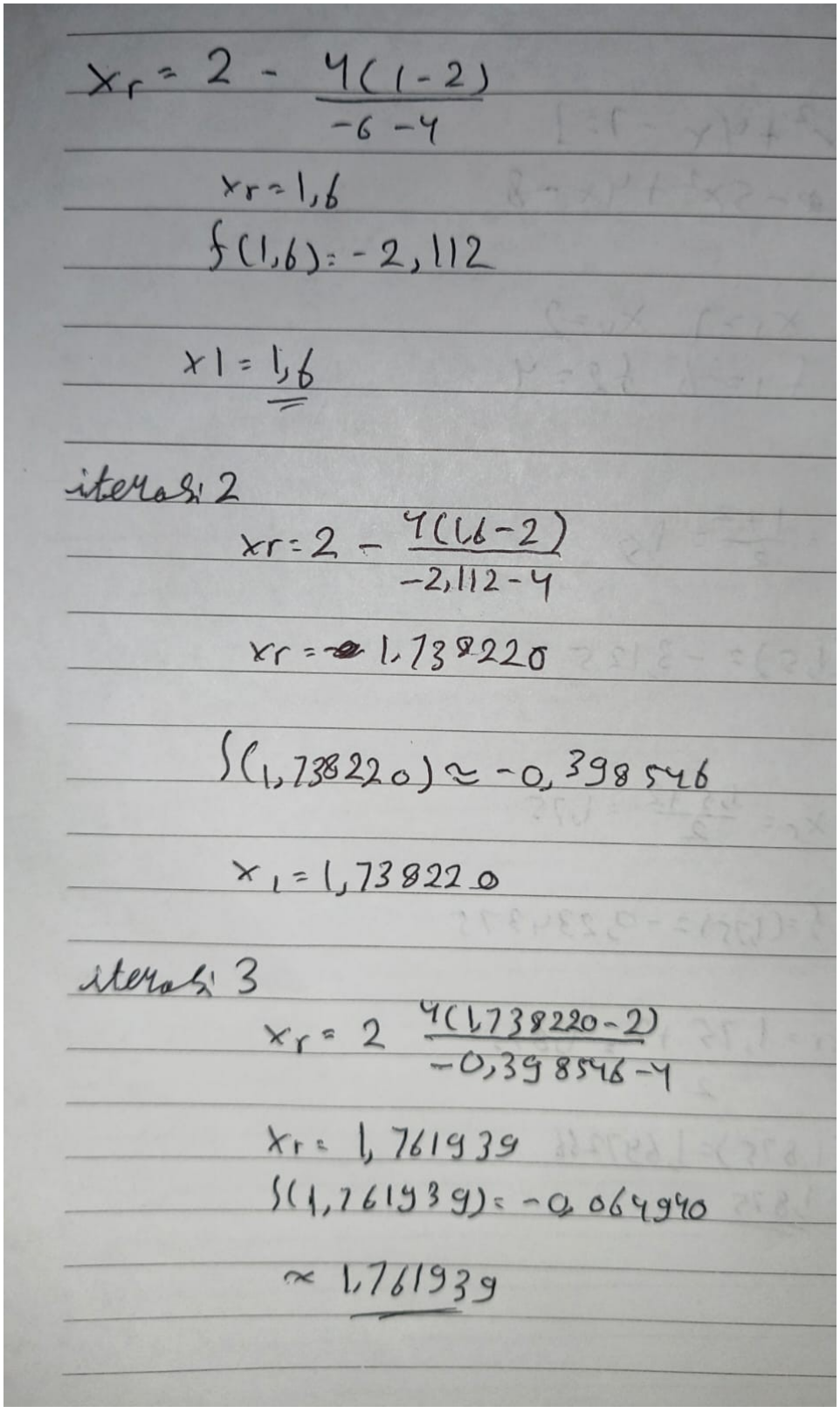
x ≈ 1,875

Pembahasan Biseksi

Pada iterasi 1 dan 2, nilai $f(x_r)$ negatif, sehingga batas bawah yang bergeser naik. Pada iterasi 3 nilainya berubah menjadi positif, artinya akar sudah terlewati dan batas atas yang diturunkan. Perubahan tanda ini menunjukkan akar berada di antara 1,75 dan 1,875.

Panjang interval selalu menyusut setengahnya: dari 1, menjadi 0,5, 0,25, lalu 0,125. Error relatif juga turun dari 14,29% menjadi 6,67%, jadi konvergensinya pasti, tetapi lambat dan bertahap.

1.REGULA FALSI PERSAMAAN 1


$x_r = 2 - \frac{4(1-2)}{-6-4}$
$x_r = 1,6$
$f(1,6) = -2,112$
$x1 = 1,6$
iterasi 2
$xr = 2 - \frac{4(1,6-2)}{-2,112-4}$
$xr = 1,738220$
$f(1,738220) \approx -0,398546$
$x_1 = 1,738220$
iterasi 3
$x_r = 2 \frac{4(1,738220-2)}{-0,398546-4}$
$x_r = 1,761939$
$f(1,761939) = -0,064940$
$\approx 1,761939$

Iterasi 1

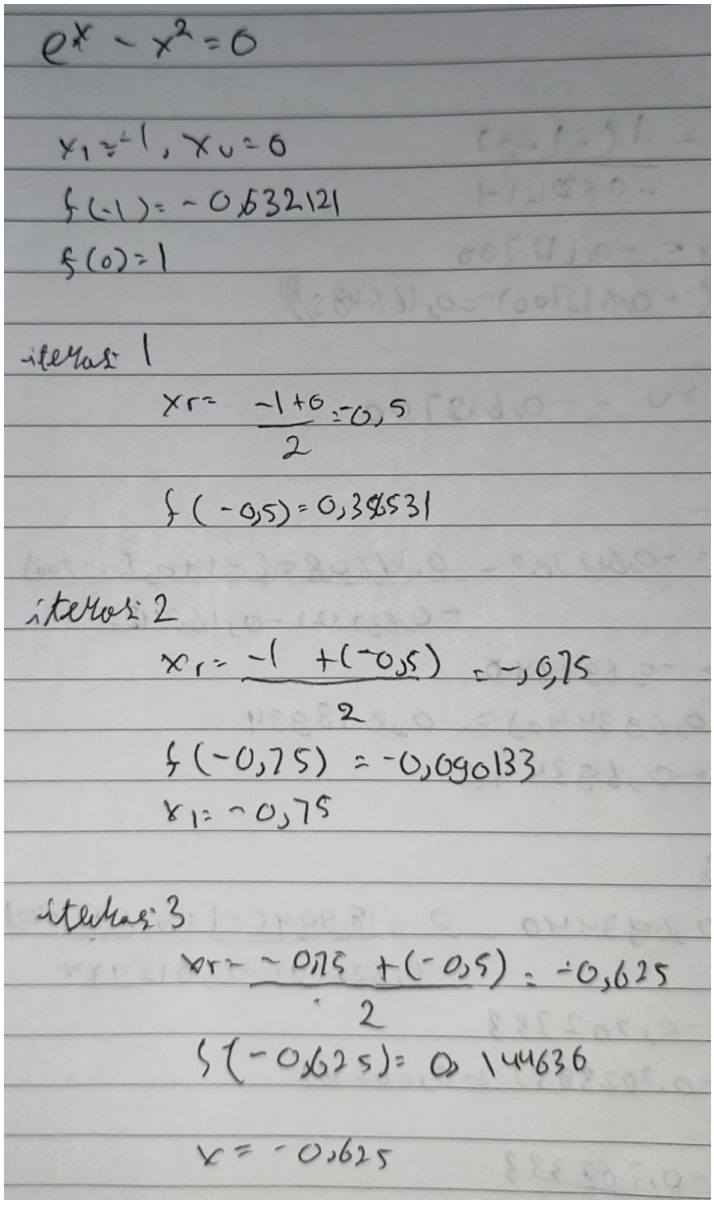
Iterasi 2

Iterasi 3

Hasil

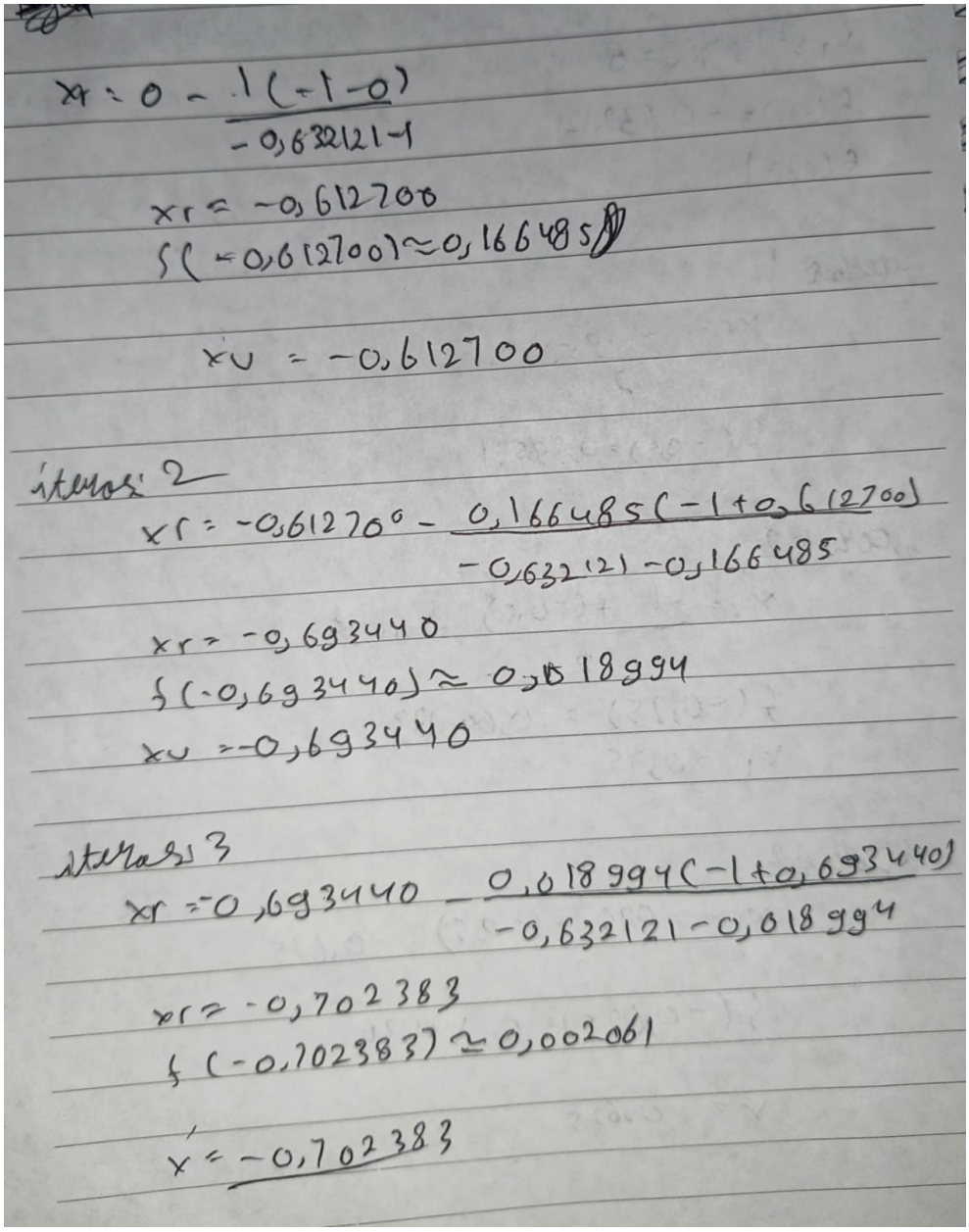
$x \approx 1{,}761939$

3.BISEKSI PERSAMAAN 2



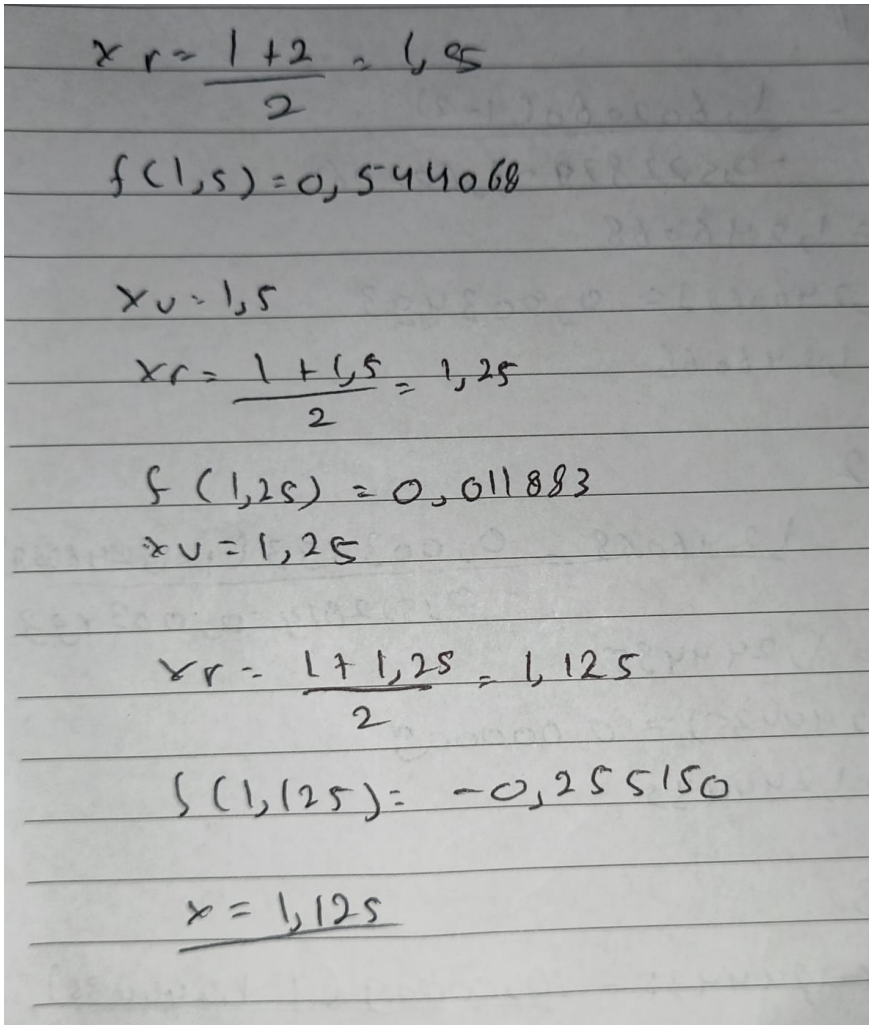
Pada metode Biseksi, interval awal [−1, 0] dipilih karena f(−1) = −0,632121 bernilai negatif dan f(0) = 1 bernilai positif, sehingga terdapat akar di dalam interval tersebut. Pada iterasi pertama diperoleh xr = −0,5 dengan f(xr) = 0,356531. Karena nilainya positif, batas atas diubah menjadi −0,5. Pada iterasi kedua diperoleh xr = −0,75 dengan f(xr) = −0,090133, sehingga batas bawah diubah menjadi −0,75. Pada iterasi ketiga diperoleh xr = −0,625 dengan f(xr) = 0,144636. Dengan demikian, setelah tiga iterasi diperoleh perkiraan akar sebesar −0,625000.

4.REGULA FALSI PERSAMAAN 2



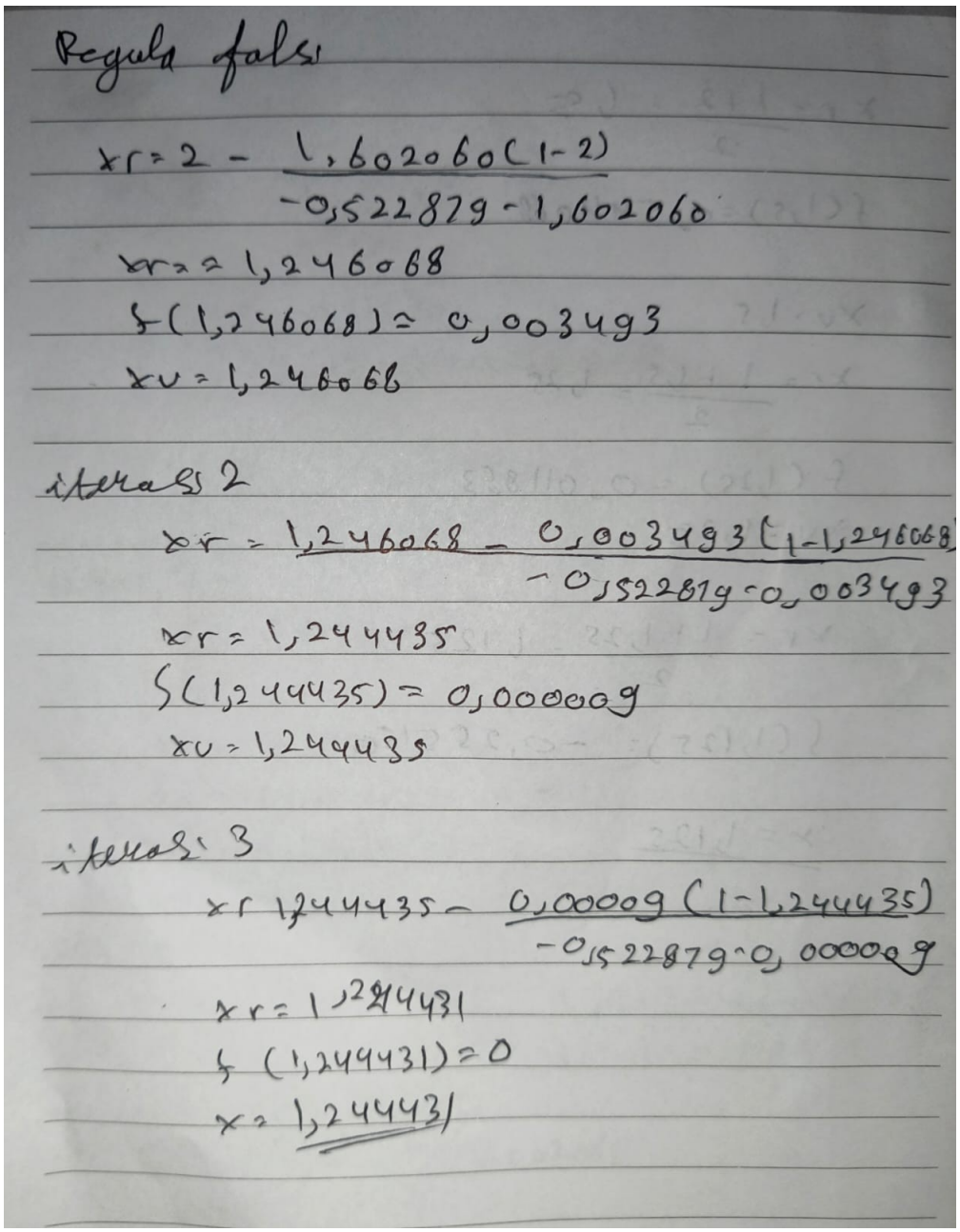
Pada metode Regula-Falsi, interval awal [−1, 0] digunakan karena f(−1) = −0,632121 dan f(0) = 1 memiliki tanda yang berbeda. Pada iterasi pertama diperoleh xr ≈ −0,612700 dengan f(xr) ≈ 0,166485, sehingga batas atas menjadi −0,612700. Pada iterasi kedua diperoleh xr ≈ −0,693440 dengan f(xr) ≈ 0,018994, sehingga batas atas menjadi −0,693440. Pada iterasi ketiga diperoleh xr ≈ −0,702383 dengan f(xr) ≈ 0,002061. Nilai fungsi semakin mendekati nol, sehingga nilai xr semakin mendekati akar. Dengan demikian, perkiraan akar setelah tiga iterasi adalah −0,702383.

5.BISEKSI PERSAMAAN 3



Pada metode Biseksi, interval awal [1, 2] digunakan karena f(1) = −0,522879 bernilai negatif dan f(2) = 1,602060 bernilai positif. Pada iterasi pertama diperoleh xr = 1,5 dengan f(xr) = 0,544068, sehingga batas atas menjadi 1,5. Pada iterasi kedua diperoleh xr = 1,25 dengan f(xr) = 0,011883, sehingga batas atas menjadi 1,25. Pada iterasi ketiga diperoleh xr = 1,125 dengan f(xr) = −0,255150. Dengan demikian, setelah tiga iterasi diperoleh perkiraan akar sebesar 1,125000.

6.REGULA FALSI PERSAMAAN 3



Pada metode Regula-Falsi, interval awal [1, 2] digunakan karena f(1) = −0,522879 dan f(2) = 1,602060 memiliki tanda yang berbeda. Pada iterasi pertama diperoleh xr ≈ 1,246068 dengan f(xr) ≈ 0,003493, sehingga batas atas menjadi 1,246068. Pada iterasi kedua diperoleh xr ≈ 1,244435 dengan f(xr) ≈ 0,000009, sehingga batas atas menjadi 1,244435. Pada iterasi ketiga diperoleh xr ≈ 1,244431 dengan nilai fungsi yang mendekati nol. Dengan demikian, perkiraan akar setelah tiga iterasi menggunakan metode Regula-Falsi adalah 1,244431.

BAB III

ANALISIS PRAKTIKUM

Kesimpulan dari Perbandingan Analisis Hasil Source Code dan Analisis Numerik

1. Hasil perhitungan manual dan source code Octave konsisten. Pada iterasi yang sama, nilai x_r (manual) sama dengan nilai c (Octave), begitu pula nilai $f(x_r)$ dan perubahan selangnya. Ini menunjukkan bahwa rumus yang dipakai pada perhitungan manual sudah sesuai dengan yang diprogramkan, yaitu $x_r = (x_l + x_u)/2$ untuk biseksi dan $x_r = x_u - [f(x_u)(x_l - x_u)] / [f(x_l) - f(x_u)]$ untuk regula-falsi.
2. Perbedaan hanya terletak pada jumlah iterasi. Perhitungan manual berhenti di iterasi ke-3, sedangkan Octave berlanjut sampai selisih c dengan batas selang tidak lebih dari toleransi 0,0001. Oleh karena itu nilai akhir Octave lebih akurat, dan nilai manual iterasi ke-3 baru berupa hampiran kasar.
3. Pada biseksi, hampiran iterasi ke-3 masih jauh dari akar. Contohnya Persamaan 1: 1,875000 dibanding akar 1,766418. Hal ini wajar karena biseksi hanya membagi dua selang tanpa memakai nilai fungsi, sehingga galatnya turun setengah pada setiap iterasi dan butuh 14 iterasi untuk mencapai toleransi.
4. Pada regula-falsi, hampiran iterasi ke-3 sudah sangat dekat dengan akar. Contohnya Persamaan 1: 1,761939 dibanding akar 1,766441, dan Persamaan 3: 1,244431 yang sama dengan hasil akhir. Hal ini karena regula-falsi memanfaatkan nilai $f(x_l)$ dan $f(x_u)$, sehingga garis sekan mendekati kurva dan konvergensinya lebih cepat (3 sampai 6 iterasi).
5. Kedua metode menghasilkan akar yang sama pada ketiga persamaan, yaitu $x \approx 1{,}7664$ (Persamaan 1), $x \approx -0{,}7034$ (Persamaan 2), dan $x \approx 1{,}2444$ (Persamaan 3). Nilai ini juga sesuai dengan perkiraan metode grafik. Selisih antara biseksi dan regula-falsi pada hasil akhir sangat kecil (orde 10^{-5}), sehingga keduanya valid.
6. Secara keseluruhan, perhitungan manual cocok untuk memahami langkah dan alur tiap metode, sedangkan source code Octave lebih efisien dan akurat karena dapat melakukan iterasi hingga toleransi tercapai tanpa risiko salah hitung. Dari sisi kecepatan, regula-falsi lebih unggul daripada biseksi pada ketiga persamaan, sedangkan biseksi unggul dalam kepastian batas galat karena lebar selangnya selalu berkurang setengah.